GENERALIZED QUANTUM ASSISTED DIGITAL SIGNATURE

A Major Project (Stage-II) *Report* Submitted to

Jawaharlal Nehru Technological University Hyderabad

In partial fulfillment of the requirements for the

award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted By

CH. TEJASRI	**(22E11A0510)**
MD ZIA FARAZ	**(22E11A0536)**
J. SRIKANTH	**(22E11A0520)**
MOHD AL TAMASH HUSSAIN	**(22E11A0537)**

Under the guidance of

Dr. Rama Prakasha Reddy Ch

Assistant professor, CSE Department

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BHARAT INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

Accredited by NAAC with 'A' Grade, Accredited by NBA (UG Programmes: CSE, ECE)

Approved by AICTE, Affiliated to JNTUH Hyderabad

Ibrahimpatnam-501 510, Hyderabad Telangana

APRIL 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BHARAT INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

Accredited by NAAC with 'A' Grade, Accredited by NBA (UG Programmes: CSE, ECE, EEE & Mechanical) Approved by AICTE, Affiliated to JNTUH Hyderabad

Ibrahimpatnam-501 510, Hyderabad, Telangana

Certificate

This is to certify that the major project (Stage-II) work entitled **"GENERALIZED QUANTUM-ASSISTED DIGITAL SIGNATURE**" is the Bonafide work done

By

CH. TEJASRI	**(22E11A0510)**
MD ZIA FARAZ	**(22E11A0536)**
J. SRIKANTH	**(22E11A0520)**
MOHD AL TAMASH HUSSAIN	**(22E11A0537)**

In the Department of Computer Science and Engineering, **BHARAT INSTITUTE OF ENGINEERING AND TECHNOLOGY,** Hyderabad is submitted to **Jawaharlal Nehru Technological University, Hyderabad** in partial fulfillment of the requirements for the award of **B.Tech** degree in **Computer Science and Engineering** during **2025-2026.**

Guide:
Dr. Rama Prakasha Reddy Ch
Assistant Professor, Project Coordinator
Dept of CSE,
Bharat Institute of Engineering and Technology
Hyderabad – 501 510

Academic Incharge:
Dr. Velmurugan P
Assistant Professor
Dept of CSE,
Bharat Institute of Engineering and Technology,
Hyderabad – 501 510

Viva-Voce held on……

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success.

We avail this opportunity to express our deep sense of gratitude and hearty thanks to ***Shri CH. Venugopal Reddy***, *Chairman of BIET*, for providing congenial atmosphere and encouragement.

We would like to thank ***Prof. G. Kumaraswamy Rao***, *Director, Former Director & O.S. of DLRL Ministry of Defence, and* ***Dr. Veerla Joshua Jaya Prasaad,*** Admin Coordinator, **Mr. Ariz Imam**, Admin Incharge for having provided all the facilities and support.

We would like to thank our Academic Incharge ***Dr. Velmurugan P***, *Assistant Professor of CSE,* for his expert guidance and encouragement at various levels of Project.

We are thankful to our Project Supervisor ***Dr. Rama Prakasha Reddy Ch***, *Assistant Professor*, Computer Science and Engineering for his support and cooperation throughout the process of this Project.

We are thankful to Project Coordinator ***Dr. Rama Prakasha Reddy Ch***, *Assistant Professor*, Computer Science and Engineering for his support and cooperation throughout the process of this project.

We place highest regards to our Parent, our Friends and Well-wishers who helped a lot in making the report of this project.

DECLARATION

We hereby declare that this Major Project (Stage-II) Report is titled "**GENERALIZED QUANTUM-ASSISTED DIGITAL SIGNATURE**" is a genuine project work carried out by us, in **B.Tech** (**Computer Science and Engineering**) degree course of **Jawaharlal Nehru Technology University Hyderabad**, and has not been submitted to any other course or university for the award of my degree by us.

Signature of the Student's

CH. TEJASRI

MD ZIA FARAZ

J. SRIKANTH

MOHD AL TAMASH HUSSAIN

ABSTRACT

With the rapid progress of quantum computing, many classical cryptographic techniques, especially digital signatures, face the risk of becoming obsolete. Traditional schemes like RSA and ECC are highly vulnerable to quantum algorithms such as Shor's, which can break their security assumptions. To address this challenge, our project explores the concept of a Generalized Quantum-Assisted Digital Signature (GQADS). Unlike classical approaches, QADS combines the strengths of quantum mechanics with existing cryptographic primitives to provide stronger security guarantees. The system leverages quantum techniques such as quantum key distribution and entanglement-based verification, while still maintaining compatibility with classical systems. This hybrid approach ensures that essential properties like authentication, integrity, and non-repudiation remain secure even against quantum adversaries. Through simulation using Python for the classical part and Qiskit for the quantum layer, project aim to design, test, and analyze the feasibility of QADS as a future-ready solution for secure communication in domains such as banking, healthcare, defense, and blockchain. The outcomes of this work not only demonstrates how quantum assistance can not only extend the life of digital signatures but also provide a practical path towards post-quantum security.

Keywords: Quantum Cryptography, Digital Signatures, Quantum Key Distribution, Post-Quantum Security, Qiskit.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF SYMBOLS AND ABBREVIATIONS

Sl.no	SYMBOL	ABBREVIATION
1	DS	Digital Signature
2	QKD	Quantum Key Distribution
3	QDS	Quantum Digital Signature
4	GQDS	Generalized Quantum Digital Signature
5	SLH-DSA	Stateless Hash-Based Digital Signature Standard
6	QADS	Quantum-Assisted Digital Signature
7	HPPK	Homomorphic Polynomial Public Key
8	NIST	National Institute of Standards and Technology
9	ECDSA	Elliptic Curve Digital Signature Algorithm

1. INTRODUCTION

1.1 Quantum Key Distribution (QKD):

Information technologies for remote data exchange are fundamental tools in modern society for managing relationships among people, legal entities or even machines. However, no matter how outstanding the technology development is, it would have been useless without cryptographic primitives able to protect our data from potential malicious entities trying to take advantage of confidential communication or dishonest personification. Confidentiality is managed by encryption and decryption algorithms using a secret key shared by the intended partners, usually denoted by Alice (the sender) and Bob (the recipient), whereas Eve generally denotes a malicious eavesdropper. Another powerful cryptographic primitive is Digital Signature (DS), which guarantees authenticity (the message originates from the claimed sender, Alice), integrity (the message is not altered by a forwarder, Bob) and non-repudiation (the sender cannot deny having sent the message because a verifier, e. g. Charlie, can recognize his signature on it). The above sentences have a probabilistic meaning: the probability of dishonest events (fake, forgery, repudiation) can be kept as small as desired in an Information-Theoretically secure (IT-secure) scenario. At present, most available DSs are not IT-secure, having a security level based on the alleged computational difficulty of solving a class of problems for which, once the solution is provided, verification is manageably accomplished in an acceptable time and proves that the solution giver was the legitimate author of the riddle as well. The recent advent of Quantum Technologies provided the appealing ability to generate IT-secure keys and share them between two remote parties. IT-security directly follows from the laws of nature embedded in the Quantum Mechanics framework, which states the true randomness in the collapse of the wave function and the non-clonability of the physical system carrying information between two parties. The almost null chance for a third party (Eve) to remain unnoticed while gleaning information from photons travelling in the quantum channel makes the secret key shared by the two parties (Alice and Bob) IT-secure against violations: a process of this kind is called Quantum Key Distribution (QKD).

1.2 Quantum Digital Signature (QDS):

On the other side, since passable scalable quantum computers may become soon available, Quantum Technologies represent a daunting threat as well, because Shor's algorithm, taking advantage of quantum logic-gate operations, proved to be able to speed up the solution of discrete logarithm and factoring problems, on which present DS implementations built their computational security. For this reason, it is imperative to develop alternative DS methods, possibly relying on IT-secure implementations or, at least, post-quantum secure methods for which no faster algorithms have been devised (yet). A Quantum Digital Signature (QDS) was first proposed in 2001 in the seminal paper [1], whose work can be thought of as an extension of a concept previously developed by Lamport in [2], where he conceived a one-time signature scheme based on a one-way function; it is a function for which knowing the output (the image) gives no clue on which was the input (the pre-image). Among the merits of Lamport's one-time signature is that the signer (Alice) can generate the signature, destroy the key used in the process (the private key, the pre-image) and no longer interact with the verifiers: they only need the output of the one-way function (the public key, the image), anytime available from a trusted repository, and, with an ingenious method involving another one-way hashing function to firmly bond the particular message to the appropriate subset of keys to be verified, neither forgery nor repudiation is possible, making transferability feasible regardless of any inconvenience that may occur to Alice after the signature accomplishment, like loss of keys, theft or even death.

The main idea in Gottesman and Chuang's work [1] was to use non-orthogonal states as "quantum one-way functions", which are guaranteed to be noninvertible by laws of quantum mechanics, even with unlimited computational power. The classical description of the state, which is only known to who prepares it (Alice), plays the role of the private secret key, whereas the quantum state itself plays the role of the public key with probabilistic test results, which cannot be tampered with without being spotted. The most striking feature of [1] is the development of the double-threshold verification method. Since the exchange of quantum states is a one-to-one process which involves the signer (Alice) and every possible verifier, a distribution phase is mandatory: all the verifiers, at least two, Bob (a first verifier with forwarding capability and possible commitment) and Charlie (a second verifier with impartiality and arbitrating intention) need to interact with Alice on a quantum

channel to get the public key (a sequence of quantum states). Moreover, to confirm Alice provided them with the same sequence of quantum states, the verifiers need to interact in a one-to-one quantum channel with the so-called "swap test" that leaves the states undisturbed (if identical) or fails probabilistically (if different). Indeed, assuming a malicious Alice, the only means to confine her probability of success in repudiation is digging a trench between two thresholds and letting in almost all the repudiation attempts (probabilistically), as explained in the following. After the sign phase performed by Alice autonomously, any time after the distribution phase, the verification phase will provide three possible outcomes in the comparison of the signature after-effect with the public key available to the verifier:

- (1-ACC) it is almost certainly Alice's signature: the detected error rates are less than the hard threshold, whose value must be compatible with system non-idealities and background noise;
- (0-ACC) it is not confidently possible to exclude that it is Alice's signature, because of a sufficient degree of similarity: the detected error rate is more than the hard threshold but less than the soft one;
- (REJ) it is almost certainly not Alice's signature on the message: the detected error rate is more than the soft threshold, nonetheless, hard enough to cope with the best forger strategy attempts.

The DS implementation in [1] is affected by many practical restrictions:

1) the need for quantum computation capabilities for all the verifiers (to perform the swap test both in distribution and verification phases);

2) the need for a quantum memory to store the public key (the sequence of quantum states describing it);

3) the need for authenticated quantum channels between all the participants,

4) finally, the lack of "universal verifiability" because only parties that have participated to the distribution phase can verify the signature.

A first step toward more feasible QDS implementations has been proposed in [3], where a replacement for the swap test has been introduced, which uses coherent states of light and realizable spatially separated multiport devices able to perform the needed quantum state comparisons, both in the distribution and verification phases.

A second important step has been proposed in [4], where the need for a quantum memory has been

circumvented by replacing the quantum public key with a classic verification key, which is no longer the same for all the verifiers, even in the ideal case, due to the random choice of the base used in the "unambiguous state discrimination" process that probabilistically extracts conclusive outcomes on some verification key elements (note that we no longer call them public keys, being different for Bob and Charlie).

A third fundamental step has been proposed in [5], where the replacement of the comparison test with a much more feasible "symmetrizing" exchange in the distribution phase leaves Alice without clues on how his private key information has been shared among the verifiers, and no means to fool one of them, but the random way, whose probability of success can be kept under control with the double-threshold method. The most significant feature of [5] is that the very same physical devices used for QKD are used to implement the QDS. Moreover, like in the QKD framework, the issue of authenticated quantum channels can be solved at the system level by "sacrificing" part of the communicated qubits on tamper tests and relying on pre-shared secrets between the intended partiesHowever, the length of the QDS is almost 2 billion qubits, and the security strength is reported to be 0.01% (one dishonest attempt of breaking the QDS protocol out of ten thousand succeeds, on average). Following the guidelines described in [7] for the implementation of a generic modern QDS protocol that does not require quantum memory, that can be realized with the same system devices used for QKD, and that does not require further assumption on the quantum channels used, reference [8] proposes a new Quantum-assisted Digital Signature (QaDS). In this paper, we propose Generalized QaDS (GQaDS), which is both a generalization and an improvement of QaDS, in many respects. In particular, we list below the main innovations we have introduced.

- We introduce an extended analysis of the security against forging of the QaDS protocol, considering also a trial-and-error approach taken by Bob when attempting his forgery.
- We propose a semi-analytical approach for the optimization of QaDS, where the sensitivity of the QaDS performance with respect to parameters such as the verification thresholds and the fraction of shared key blocks between Bob and Charlie is assessed and exploited in the design of GQaDS.
- We replace hash functions in the QaDS implementation with Carter-Wegman Message Authentication Codes (MACs), obtaining the double result of strengthening the scheme security and reducing the signature lengths.

- For particular scenarios where the second verifier (Charlie) is an authority endowed with a safe reputation, we propose a simplified version of GQaDS, named deterministic GQaDS, which further reduces the requirements in terms of signature length.

In section II the QaDS proposed in [8] is briefly summarized, and the main scheme parameters are introduced. In Section III, the security of QaDS is discussed, with a more advanced discussion on the security against forgery. Section V describes GQaDS, especially its novelties with respect to QaDS, and deterministic GQaDS.

1.3 Introduction to QISKIT:

Qiskit as a pivotal tool in the evolving landscape of quantum information science, with a focus on its integration into hybrid cryptographic systems like the Quantum-Assisted Digital Signature (QADS) framework outlined in this B.Tech major project. Drawing from authoritative sources, we explore Qiskit's historical development, technical architecture, practical utilities, and specific synergies with quantum-resistant cryptography. This analysis underscores Qiskit's role in academic simulations, where it enables rigorous testing of quantum threats without the barriers of hardware access, fostering innovation in high-stakes fields such as financial security and IoT defense.

Qiskit's layered design ensures flexibility for both novices and experts. At the base, Qiskit Terra provides circuit composition tools, supporting gate-level abstractions and pulse-level control for hardware-aware simulations. Qiskit Aer stands out for noise-aware emulation, capable of full state-vector simulations up to 44 qubits on GPU clusters like LUMI, which is vital for modeling large-scale quantum attacks in our QADS project. The Qiskit Nature module extends to quantum information science, including operators for entanglement detection key to QADS verification protocols. For hybrid systems, Qiskit excels in orchestration: plugins like QRMI standardize resource management across providers (e.g., IBM, IonQ, Amazon Braket), while Qiskit Serverless enables distributed quantum-classical executions on multi-cloud setups. Algorithmic add-ons, such as those for Shor's and Grover's algorithms, facilitate direct simulation of cryptographic disruptions, as demonstrated in tutorials breaking RSA on simulated quantum computers.

This table highlights Qiskit's modularity, enabling the project's dual focus: theoretical analysis of quantum vulnerabilities and practical prototyping of resilient signatures.

2. RELATED WORK

“FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA)”

NIST, 2024. NIST in its publication FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA) proposed a stateless hash-based digital signature algorithm specifically designed for post-quantum security applications. The work utilizes the SPHINCS+ framework, which employs hypertree, WOTS+, and FORS constructions to generate strong quantum-resistant signatures. Unlike number-theory-based algorithms (e.g., RSA, ECC) that are vulnerable to quantum attacks, SLH-DSA relies on hash-based constructions to ensure future-proof digital authentication. [1].

“FIPS 186-5: Digital Signature Standard (DSS)”

National Institute of Standards and Technology (NIST), 2023.The National Institute of Standards and Technology (NIST) in its FIPS 186-5: Digital Signature Standard (DSS) defines an updated set of digital signature algorithms used for authenticating identity and maintaining data integrity in secure communication systems. The standard includes algorithmic specifications such as RSA, EC-DSA, and EdDSA, along with domain parameter recommendations. The revision focuses on deprecating older algorithms like DSA while standardizing advanced elliptic-curve-based signatures to enhance cryptographic performance. This work represents a major modernization effort in global cryptographic practices, establishing a strong foundation for secure digital transactions and electronic identity management [2].

“Additional Digital Signatures – First Round Status Report” (NIST IR 8528)

G. Alagic, Z. Bajcsy, L. Bassham, L. Chen, M. Dworkin, S. Kerman, A. Regenscheid (2024)

G. Alagic, Z. Bajcsy, L. Bassham, L. Chen, M. Dworkin, S. Kerman, and A. Regenscheid in NIST IR 8528: Additional Digital Signatures – First Round Status Report presented a detailed evaluation of candidate post-quantum digital signature schemes submitted for NIST’s standardization process. The authors analyzed 40 first-round submissions and selected 14 based on their mathematical soundness, computational efficiency, and resistance to quantum attacks. The study revealed the broadening of the post-quantum cryptographic (PQC) algorithm pool, with innovative mathematical approaches extending beyond traditional lattice-based schemes. This report significantlycontributed

to shaping the direction of ongoing PQC standardization efforts and expanding the diversity of cryptographic primitives available for future digital security [3].

"A Novel Post-Quantum Secure Digital Signature Scheme Based on Neural Network"

S. Kumar and M. Arzoo Jamal, 2025

S. Kumar and M. Arzoo Jamal introduced an innovative approach in A Novel Post-Quantum Secure Digital Signature Scheme Based on Neural Network. Their research integrates neural-network-based architectures with multivariate polynomial signature schemes to enhance resistance against quantum adversaries. The proposed method employs binary-weight neural networks that strengthen signature generation and verification mechanisms, ensuring existential unforgeability under chosen-message attacks (EUF-CMA). This hybrid approach bridges artificial intelligence and cryptographic design, offering a novel and adaptive framework for post-quantum digital signature systems. The study demonstrates that AI-assisted models can play a vital role in creating flexible, secure, and efficient post-quantum cryptographic standards [4].

"Benchmark Performance of Homomorphic Polynomial Public Key Cryptography for Key Encapsulation and Digital Signature Schemes"

R. Kuang, M. Perepechaenko, D. Lou, and B. Tank 2024

Finally, R. Kuang, M. Perepechaenko, D. Lou, and B. Tank [5] presented a comprehensive benchmarking study titled Benchmark Performance of Homomorphic Polynomial Public Key Cryptography for Key Encapsulation and Digital Signature Schemes. This work benchmarks a new cryptographic model—Homomorphic Polynomial Public Key (HPPK)—which supports both encryption and digital signature functionalities. Through extensive testing across various key sizes and platforms, the authors demonstrate that HPPK achieves compact key/signature sizes and high computational efficiency. The scheme is particularly suited for IoT and resource-limited environments. Additionally, its homomorphic property supports secure computation over encrypted data, thereby enhancing privacy-preserving cryptographic applications in cloud and distributed systems.

Collectively, these studies contribute significantly to the development of quantum-resistant digital signature schemes. They explore a wide range of mathematical foundations—hash-based, elliptic curve, multivariate, neural network, and homomorphic polynomial cryptography—each aiming to ensure strong data authentication and integrity in a post-quantum world. These works provide a

comprehensive foundation for advancing research in AI-assisted and post-quantum cryptographic frameworks.

Enhancing Blockchain-Based Digital Signatures

A. Gupta and R. Singh, 2020.

This study focuses on improving the efficiency and integrity of digital signatures used within blockchain environments. Gupta and Singh [6] highlight that traditional blockchain signature mechanisms, while secure, often suffer from computational delays and resource overhead. Their work introduces a lightweight enhancement layer integrated with SHA-256 and Hyperledger Fabric to accelerate signature processes without compromising security. Through Python-based modeling and performance evaluation, the authors demonstrate significant reductions in signing and verification time. Their contribution is important because it shows how enhancing signature operations can directly improve blockchain transaction throughput, scalability, and user experience especially in applications such as digital identity, smart contracts, and secure financial exchanges.Their enhanced model reduces computational bottlenecks, which is crucial for real-time and high-frequency blockchain transactions. The study highlights that the proposed lightweight signature layer allows block validation to occur more smoothly, reducing network latency and increasing overall trust in distributed systems. Their findings encourage further exploration of hybrid blockchain-signature models, especially for financial institutions and supply-chain systems moving toward distributed digital ecosystems.

Lightweight Post-Quantum Cryptography for IoT Devices

L. Chen, H. Park, and Y. Zhao, 2021. Chen and colleagues investigate the suitability of post-quantum digital signature schemes for low-power Internet of Things (IoT) systems. Recognizing that IoT devices often struggle with limited memory, energy, and processing capabilities, the authors analyze the performance of lattice-based NTRU signatures using MATLAB simulation tools. Their results show that certain post-quantum schemes maintain strong security while remaining lightweight enough for constrained environments. The study emphasizes the need for quantum-safe solutions that do not overload small embedded devices. This work is valuable because it demonstrates that secure post-quantum communication in IoT networks is achievable without sacrificing energy efficiency.The authors further discuss scalability concerns, noting that IoT deployments often involve thousands of interconnected devices, each requiring efficient cryptographic operations. Their

experiments show that NTRU-based signature schemes maintain predictable performance even as node counts increase. Moreover, they address real-world applicability by testing their model under fluctuating network and power conditions, demonstrating that quantum-resistant signatures can be deployed without modifying IoT hardware [7].

Improved ECDSA Performance for Mobile Environments

M. Rahman and S. Devi, 2022.Rahman and Devi aim to optimize the performance of the Elliptic Curve Digital Signature Algorithm (ECDSA) on mobile devices, where battery life and processing power are major constraints. Using OpenSSL libraries integrated within Android Studio, they evaluate existing bottlenecks in signing and verification processes. The authors then introduce code-level and parameter-level optimizations that significantly reduce power consumption and processing time. Their findings show that secure mobile applications can maintain strong cryptographic protection while running more efficiently, making this study highly relevant for secure messaging apps, mobile banking, and IoT gateways that rely on mobile hardware.

In addition to performance gains, Rahman and Devi examine user-experience implications, noting that faster and more energy-efficient cryptographic operations contribute to smoother application behavior and reduced device heating. Their optimized ECDSA model also enhances authentication speed, making mobile payments and secure access systems more responsive. By benchmarking multiple Android devices, they illustrate that their improvements are not limited to specific hardware. Their research ultimately demonstrates that traditional cryptographic algorithms can remain competitive in modern mobile systems when carefully tuned, providing a secure and efficient foundation for next-generation mobile applications [8].

Efficient Hash-Based Signature Schemes for Long-Term Data Protection

T. Nakamura and K. Ito, 2023. Nakamura and Ito explore hash-based signatures, specifically the SPHINCS+ framework, as a sustainable long-term security solution resistant to quantum threats. Unlike lattice-based systems, hash-based signatures have deterministic security grounded in cryptographic hash functions rather than complex mathematics. Using C/C++ implementations, the authors demonstrate their stability, strong security guarantees, and suitability for long-term archival systems where data integrity must remain intact for decades. Their research asserts that hash-based approaches may be the safest option for extremely long data lifespans. Their results reveal that

SPHINCS+ offers reliable performance despite large signature sizes, making it practical for applications where verification stability outweighs storage constraints. The authors argue that hash-based systems provide unmatched predictability because their security depends only on the strength of hash functions, not on assumptions that may weaken with future mathematical breakthroughs. Their work advocates for adopting hash-based signatures as the foundation for secure long-term storage infrastructures [9].

Neural-Network-Assisted Post-Quantum Signature Generation.

S. Miller and P. Zhang, 2025. Miller and Zhang present an innovative hybrid approach that combines neural networks with post-quantum signature generation. Leveraging Python, TensorFlow, and multivariate cryptographic structures, they design a model that accelerates signature generation while preserving post-quantum resistance. Their framework shows notable improvements in computation time, suggesting that machine learning can enhance traditional cryptographic workflows. This study opens a promising research direction where artificial intelligence contributes directly to the design of faster and more adaptive post-quantum digital signature schemes. Miller and Zhang extend their analysis by exploring how neural networks can adapt signature behavior based on workload patterns, opening possibilities for dynamic optimization. Their framework demonstrates the potential for AI-assisted cryptography, where models can learn to minimize redundant computations and predict optimal parameters for signature generation. They also highlight the benefits for resource-constrained environments such as mobile devices or IoT systems where fast computation is essential. Their study suggests that combining neural networks with post-quantum cryptographic algorithms could become a transformative approach, enabling faster, smarter, and more efficient digital signature systems in the near future [10].

3. MOTIVATION

The world is moving rapidly towards quantum computing, and this shift brings both powerful opportunities and serious security risks. Classical cryptographic systems such as RSA, Diffie Hellman, and ECDSA form the backbone of today's secure communication, digital signatures, online transactions, and identity verification. These systems were built for classical computers, but new advancements in quantum algorithms, especially Shor's Algorithm, have shown the potential to break them with relative ease. What once seemed theoretically possible is now becoming a practical concern as quantum processors continue to evolve, increasing in qubit capacity, stability, and efficiency. This creates an urgent need to rethink how we protect data. Modern digital platforms ranging from banking to healthcare, cloud services, government databases, and IoT devices cannot rely solely on cryptographic methods that may soon become obsolete. To stay secure in the post-quantum era, systems must transition to authentication methods that remain strong even when facing quantum-capable adversaries. In this context, Post-Quantum Cryptography (PQC) and quantum-enhanced authentication models offer a promising direction for designing robust future-proof security solutions.

At the same time, cyber-attacks are becoming more advanced and harder to detect. Attackers are using sophisticated techniques to intercept keys, tamper with messages, forge signatures, or impersonate users. Ensuring data integrity and authenticity has never been more important. Quantum technologies offer unique advantages that traditional systems simply cannot match. For instance: Quantum Randomness produces genuinely unpredictable values, making forgery or token duplication extremely difficult. Quantum Key Distribution (QKD) immediately detects any attempt to intercept communication because quantum states collapse when observed. This makes man-in-the-middle attacks and key theft almost impossible. Quantum-assisted verification methods improve the overall trustworthiness of authentication processes by identifying even the smallest irregularities. Together, these innovations create an environment where digital communication can achieve levels of security that were previously unimaginable with classical methods alone. By integrating quantum principles with classical cryptographic frameworks, the proposed system aims to build a next-gen.

4. OBJECTIVES

- To design a quantum-supported digital signature model that integrates classical cryptography with quantum technologies.
- To enhance data authenticity, message integrity, and resistance to quantum attacks.
- To utilize QKD-based key generation for improved security.
- To conduct simulations using Qiskit for quantum-assisted verification.
- To compare GQADS performance against traditional RSA and ECDSA signatures.
- To provide a scalable, future-proof model adaptable to cloud, IoT, and distributed systems.

4.1 FEASIBILITY STUDY

The feasibility study evaluates whether the proposed **Generalized Quantum-Assisted Digital Signature (GQADS)** system can be developed, deployed, and maintained effectively. This phase helps ensure that the project is realistic, practical, and beneficial to the organization or institution undertaking it. By analyzing the economic, technical, and social aspects, we verify that the system does not impose unnecessary burdens and can operate smoothly within available constraints. Understanding these parameters provides a clear roadmap for successful project development and implementation.

Three key considerations involved in the feasibility analysis are

ECONOMICAL FEASIBILITY

The economic feasibility examines whether the proposed GQADS system is cost-effective and financially viable. Since the system uses **open-source tools such as Python, Qiskit, NumPy, and IBM Quantum simulators**, the overall development cost remains significantly low. Only minimal expenditures may arise from optional cloud-based quantum runtime services if the system is tested on real quantum hardware.

System Costs:

1. Development Costs

- Hardware and software procurement

- Setting up Python, Qiskit, and development environment
- Optional access to premium quantum computing cloud nodes

2. Production Costs

- Operations and maintenance of the system
- Minimal manpower for updates and testing
- Usage of local hardware and open-source software

3. System Benefits

- Reduced computational overhead compared to classical-only schemes
- Lower operational cost due to hybrid classical-quantum optimization
- Enhanced accuracy and reduced signature verification errors
- Improved long-term security against quantum threats

Overall, the economic analysis confirms that the GQADS system is well within feasible financial limits and offers strong long-term value.

TECHNICAL FEASIBILITY

The technical feasibility evaluates whether the required technologies, tools, and expertise are readily available. The proposed GQADS system leverages **quantum simulators, hybrid cryptographic algorithms, and classical digital-signature processes**, all of which can be implemented using existing and widely supported platforms. The system does not require high-end hardware because Qiskit allows simulation of quantum circuits on standard machines.The system does not require any specialized hardware, and the technical demands remain modest. Developers can build and test the entire system on classical machines using simulation, ensuring minimal resource requirements. The technology stack is stable, well documented, and accessible, making implementation smooth and practical.

SOCIAL FEASIBILITY

Social feasibility determines whether users will comfortably adopt and trust the system. Since GQADS enhances the security of digital signatures without complicating the user workflow, it is

expected to be well-accepted. Users only interact with simple interfaces while the quantum operations run in the background, reducing the risk of confusion or resistance.

Proper training, demonstrations, and documentation ensure that users can quickly understand how the system works and why it improves security. By building user confidence and addressing initial concerns, the system fosters acceptance. Users are also encouraged to share feedback, allowing continuous improvement. Since the system aims to protect user data and enhance trust in digital transactions, its acceptance level is anticipated to be high.

5. PROBLEM STATEMENT

5.1 Existing System:

In today's digital world, we use cryptographic techniques like RSA and ECDSA to secure data and verify digital identities. These methods depend on solving tough mathematical problems such as integer factorization and discrete logarithms. As long as computers can't solve these problems quickly, our data stays safe.

However, with the rapid growth of quantum computing, this assumption is no longer reliable. Shor's algorithm, for example, can easily break the mathematical foundations of these classical cryptosystems if a sufficiently powerful quantum computer is built. This means that the digital signatures we currently use to verify identities and secure communication could become insecure in the near future.

Researchers have tried to counter this by developing Quantum Digital Signatures (QDS), which use the principles of quantum mechanics to provide unbreakable, physics-based security. But there are still major challenges:

- QDS systems need special quantum hardware that's expensive and not widely available.
- They require quantum channels and even quantum memory to work.
- Verification can only be done by those who participated in the key exchange.
- Signing even a single bit of data currently needs billions of qubits, making it highly impractical for real-world use.

Because of these issues, fully quantum systems are not yet practical for everyday communication. There's a need for a more balanced and realistic approach like ours which uses the strength of quantum technology without depending entirely on it.

5.2 Proposed System:

The proposed system introduces a Generalized Quantum-Assisted Digital Signature (GQADS) model that integrates classical cryptographic mechanisms with quantum computing principles to enhance the security, authenticity, and resilience of digital signatures. As classical digital signature schemes such as RSA and ECDSA face growing vulnerabilities due to advancements in quantum computing, the GQADS system provides a hybrid architecture designed to resist future quantum attacks while maintaining efficiency and scalability.

The system operates by combining traditional key-generation, signing, and verification processes with quantum-assisted operations such as superposition, entanglement, and quantum key distribution (QKD). The quantum layer is responsible for secure key exchange and generating quantum-enhanced parameters, while the classical layer handles signature generation and message verification. This hybrid approach ensures backward compatibility with existing systems while adding a quantum-resilient layer of security.

To implement and test the system, quantum simulation tools such as Qiskit, IBM Quantum Lab, or QuTiP are used to model quantum circuits responsible for key distribution and quantum-assisted functions. Classical cryptographic algorithms are implemented in Python, enabling smooth integration between the classical and quantum components. The proposed system also uses existing cryptographic datasets and quantum simulation outputs to evaluate the performance under various conditions.

The GQADS model is designed to provide improvements in processing time, key strength, signature size, verification accuracy, and overall security robustness. By combining the strengths of classical and quantum techniques, the system offers stronger resistance to forgery, tampering, and quantum decryption attacks. It ensures secure data transfer even in environments where traditional cryptographic systems may become vulnerable. Finally, the system is scalable and can be adapted for various digital platforms, making it suitable for real-time applications in secure communication, financial transactions, cloud security, and distributed systems. The proposed GQADS system sets the foundation for next-generation digital signature technology capable of withstanding the evolving landscape of quantum-enabled cyber threats.

5.3 TIMELINE OF THE PROJECT

- Sept 2025 → Literature Review & Proposal
- Oct 2025 → System Design & Methodology
- Nov 2025 → Classical Implementation
- Dec 2025 → Quantum Implementation
- Jan 2026 → Integration & Testing
- Feb 2026 → Evaluation & Documentation
- Mar 2026 → Final Report & Submission

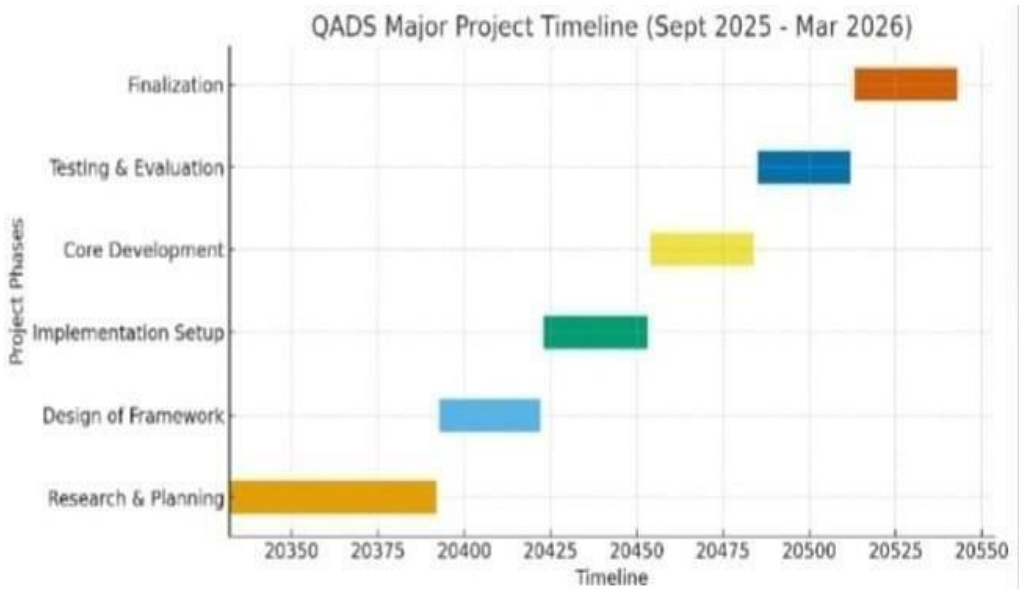


Fig.5.3: Timeline of the project

6. FUNCTIONAL REQUIREMENTS

The proposed system integrates principles of quantum cryptography with classical digital signature mechanisms to enhance security. The following are the key functional requirements of the system:

User Registration and Key Generation

- The system should allow users to register securely.
- It must generate a pair of keys: Classical public-private key pair.
- Quantum-assisted key (derived using quantum principles such as randomness or entanglement simulation).
- The private key must remain confidential and securely stored.

Message Signing

- The system should enable users to sign digital messages.
- It must combine: Classical hashing algorithms (e.g., SHA-based hashing).
- Quantum-assisted randomness for additional security.
- The generated signature must be unique for each message.

Signature Verification

- The system should verify the authenticity of a signed message.
- It must: Validate the signature using the sender's public key.
- Ensure message integrity (i.e., detect any modification).
- The output should clearly indicate whether the signature is valid or invalid.

Quantum Key Distribution Simulation

- The system should simulate quantum key distribution (QKD) principles.
- It must: Generate secure keys using quantum randomness.
- Detect potential interception attempts (simulated eavesdropping).

Security Monitoring

- The system should monitor suspicious activities.
- It must: Detect tampering or replay attacks.
- Log unauthorized verification attempts.

User Interface

- The system should provide a simple and intuitive interface.
- Users must be able to: Upload messages.
- Generate signatures.

- Verify signatures easily.

6.1.1 Minimum Software Requirements:

Operating system	: Windows 10 and above.
Technology	: Quantum computing, Cryptography (Digital Signature)
Programming Language	: Python 3.10
IDE Used	: Qiskit Framework

6.1.2 Minimum Hardware Requirements:

Processor : Intel i5 or higher

Storage : 500 GB free space

RAM : 8 GB / 16 GB

GPU : Optional NVIDIA GPU (GTX 3050 or higher)

OS : Windows / Linux / macOS Ubuntu 22.04 or Windows 11

7. SYSTEM DESIGN METHODOLOGY

7.1 PROTOCOL DESIGN:

The general structure of the proposed Q-DS protocol is divided into a distribution phase and a messaging phase. The protocol begins with the distribution phase, in which Alice and Bob carry out a QKD process, for example the well-known decoyed-states BB84 protocol [2]. This protocol is executed and the result is a secret symmetric key k1 of length lk, shared by Alice and Bob. These same steps are carried out between Alice and Charlie (a second possible receiver of the message), obtaining, at the end of the process, a secret symmetric key k2 of length lk . The keys k1 and k2 are divided into n blocks of a certain length and stored. Each block has a fixed labeling B = {B1, B2, ..., Bn} which is known by all the users. Once the generation of symmetric keys between pairs is finished, Bob and Charlie carry out a process of exchanging random blocks of keys through an encrypted and authenticated classic channel, for example, with other QKD-generated keys or hybridizing QKD with CRYSTALS-KYBER through a Key Derivation Function (KDF). For this, given Zn as the group of permutations of n elements so that |Zn| = n!, we denote the random permutation of n elements $\gamma j \in Zn$, with j = {b, c}, as

$$\gamma j(i) = ai,\ ai \in \{1, 2, ..., n\} \quad (1)$$

Bob sends to Charlie the subset of elements k′1 corresponding to the first n/2 indices of B associated with k1 after applying γb . Next, Charlie sends to Bob the subset of elements k′2 corresponding to the first n/2 indices of B associated with k2 after applying γc. Alice does not know which blocks Bob and Charlie have exchanged. This is a necessary condition so that Alice cannot repudiate the authorship of the message, as we will see later. The protocol continues with a messaging phase in which Alice generates the digital signature for a given message and sends it to the recipients, who verify the validity of both the message and the signature, as shown in Figure 1. Alice, the signer, wants to send Bob, one of the possible verifiers, a message (m) of any length and its associated signature (Sa). To do so, Alice generates a combined key ka of length 2lk by concatenating the secret symmetric key shared with Bob to the secret symmetric key shared with Charlie (ka = k1||k2). Alice calculates the hash of the message as ha = h(m). The use of a hash function allows us to take as input text of any size and returns a fixed-size digest equal to δm. Then, Alice encrypts ha with the composed secret key ka using a One-Time Pad (OTP) encryption function, whereby an element-by-element XOR is made between ha and ka, obtaining ca = ENCka (ha). In order to carry out this step, ha and ka must have the same length, so 2lk = δm. At this point, it is important to remember

that k1 and k2 were divided into n blocks each, so ka has 2n blocks. And, in turn, ca will also be made up of 2n blocks. As a last step, Alice generates individual hashes for each of the blocks of ca, each with length δB, thus obtaining the signature Sa of length 2n · δB, associated with her message.

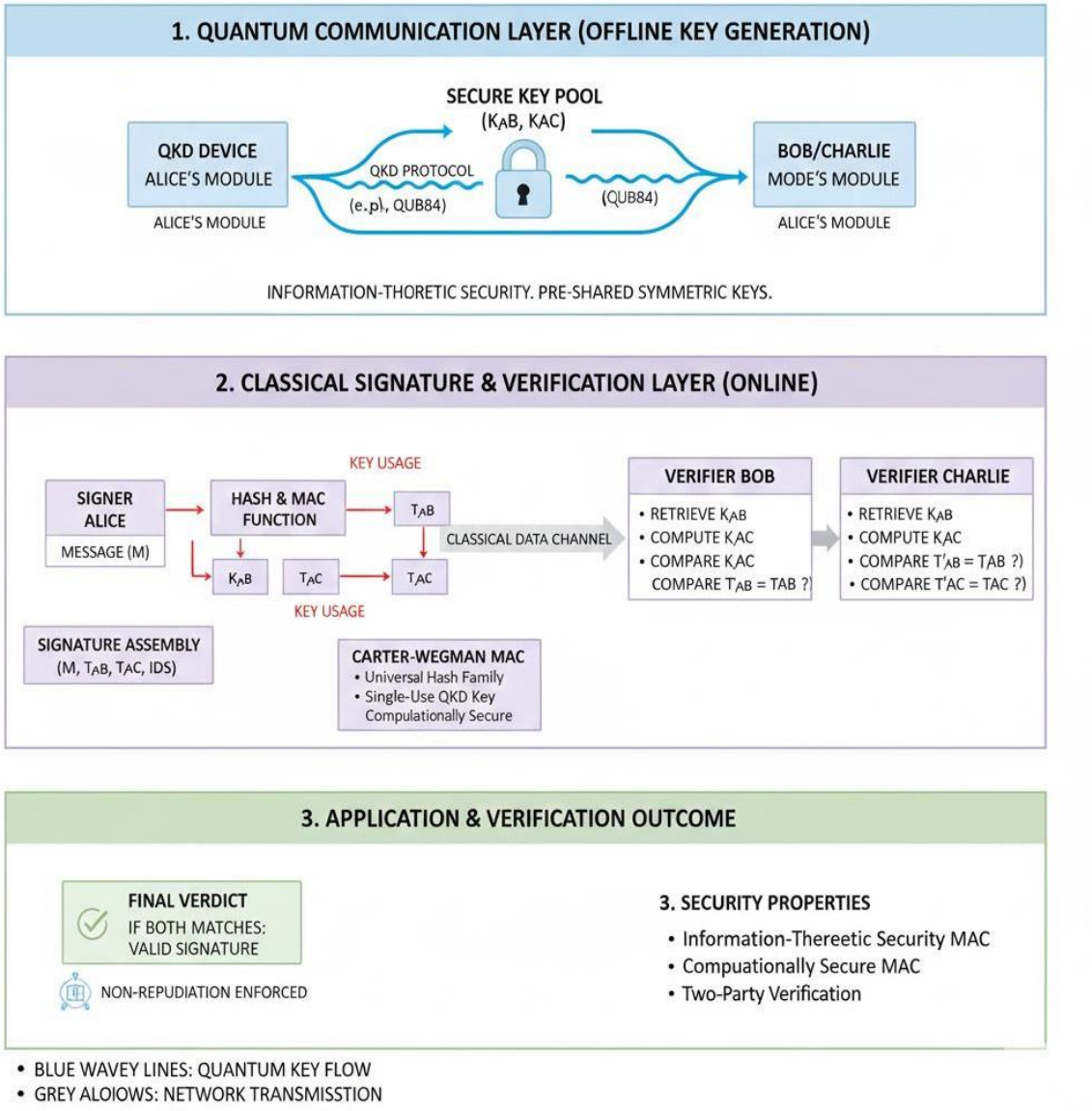


Fig 7.1: System Architecture of GQDS

7.2 SECURITY ANALYSIS:

The security of the proposed Q-DS protocol is analyzed from different perspectives. First, we assess the robustness of the Q-DS protocol against integrity attacks on the message, meaning that if m is modified along the way, it is detected and the protocol aborted. To do so, we assume that Bob is a malicious player. Thus, Alice generates the Q-DS as specified in Section 2. Bob receives (m, Sa) from Alice, and verifies and accepts the signature. After that, he modifies the message $m \rightarrow M$, but keeping the signature generated by Alice Sa. Bob sends to Charlie the tuple (M, Sa). Upon receiving it, Charlie performs the verification process, and the detection of the attack attempt is straightforward, since $Sc \neq Sa$. Modifying the message causes the first alteration in the calculation of the message hash. Since the calculation of Sa and Sc depends on the value of ca and cc, respectively, and these depend on the value of ha and hc, the alteration produced by the change in message is propagated, triggering an error in Charlie's verification test. Note that in this case, Charlie cannot distinguish whether the reason for the negative verification test was an attack on the integrity of the message or an attempt to forge the signature. In order to succeed in an integrity attack, Bob would have to be able to find a collision or a second preimage of the hash value. A collision attack means finding any two messages, m and M, with $M \neq m$, such that $h(M) = h(m)$. A preimage attack means that, from the given message digest of m, h(m), finding a different input M that provides the same hash value $h(M) = h(m)$. The difference between collision and a second preimage is that in the later, the malicious user can be unaware of the content of m if, for example, it is encrypted. These attacks can be prevented, as we will see at the end of this section. Second, we assess the robustness of the Q-DS protocol against a signature forgery attack perpetrated by Bob. In this case, Bob generates a fake digital signature Sf associated with the tampered message M. The generation process of Sf is the same as Alice's, except that Bob does not know around half of Charlie's key k2, so the key $K \neq k2$ used to generate k f will produce an alteration that will propagate through all the steps of the Charlie verification test and will result in a signature rejection. In order to successfully forge the signature, Bob would have to be able to guess the unknown elements of k2. To do this, Bob could carry out different strategies. (1) Guess the unknown bits of k2. Due to the randomness of the QKD-generated key, he has a 50% chance of matching the value of each bit. If he has to guess 0.5lk bits, where lk is the length of k2, the probability that all his guesses are correct is $Pguess = 2^{-lk/2}$. Therefore, the larger the length of the key, the smaller Pguess, as plotted in Figure 3 (left). (2) Extract the unknown elements of k2 from the signature Sa. Block hashes of a certain size prevent Bob from accessing the

full value of ca and, therefore, the keys from being extracted. (3) Find a preimage, that is, given a hash value of H, identifying the input message m that provides h(m) = H. These kind of attacks are mainly performed by brute force but can be prevented by taking into account a series of security parameters and requirements in the choice of hash functions and the input length, as is explained at the end of this section.

7.3 IMPLEMENTATION

The Q-DS protocol was implemented using C++ language integration OpenSSL libraries. This study was carried out under the Digest and Sign paradigm, whereby before signing a message, it is hashed, and the message digest is signed afterwards. This is a standard technique used for singing procedures on most applications. The system was built and run in an Oracle VM VirtualBox Ubuntu 64 bits operating system with an 11th Gen Intel(R) Core (TM) i7-1185G7 processor. For the key generation, a pair of commercial discrete variable QKD devices was used in a back-to-back setup, with a security parameter of $\epsilon = 10{-}9$. The QKD protocol used was a decoy state BB84 composed of one signal, μ0, and two decoy states, {μ1, μ2}, with intensities of {0.45, 0.225, 0} and probabilities of {0.2, 0.6, 0.2}, respectively.

The parameters that can be modified in the Q-DS protocol are the message length lm, hash function applied to the message, key length lk, number of blocks n, hash function applied to the blocks, and allowed number of blocks with error. Depending on the parameters chosen, the system will provide a specific security level and the probability of a malicious player succeeding in a repudiation attack, as specified in Section 3.

In the first part of this research, we analyzed the efficiency of the Q-DS protocol during the key generation (KeyGen) and signature generation and verification (SigGen and SigVer) when increasing lm, lk and n. The hash function applied to the message and the blocks was SHAKE256 in all cases. Subsequently, we carried out a comparative evaluation of the performance of the Q-DS with the most relevant pre-quantum and PQC DSAs.

The pre-quantum DSAs that were selected were those most widely implemented in current systems, that is, elliptic curves and Rivest–Shamir–Adleman (RSA). For a security level of 128 bits, the following algorithms were implemented: ECDSA—BrainpoolP256r1, Secp256r1 and

EDDSA—ED25519. For a security level of 256 bits, the following algorithms were implemented: ECDSA—BrainpoolP512r1, Secp521r1 and RSA15360.

The PQC digital signature algorithms corresponded to those that are standardized by NIST, which are CRYSTALS-Dilithium under the FIPS 204 standard [4], SPHINCS+ under the FIPS 205 standard [5], and FALCON, whose standard will be published at a later date.

The specific algorithms implemented were Dilithium2, Falcon512, sphincssha2128{f,s}simple and sphincsshake128{f,s}simple, for a security level of 128 bits.

In addition to, Dilithium5, Falcon1024, sphincssha2256{f,s}simple and sphincsshake256{f,s}simple, for a security level of 256 bits.

The pre-quantum and post-quantum algorithms were benchmarked using the OpenSSL generic interface, and the post-quantum implementation was provided by the Open Quantum Safe (OQS) project through the open-source C library liboqs [17]. To compare the different solutions, under an equivalent level of security, common parameters were set in all of them, such as the length of the message (lm = 10 kB), the hash function to calculate the message digest (SHAKE256(m, δm)) and the output length of the message digest (δm = 512 B), except for RSA, where δm = 256 B.

The security levels of pre-quantum and post-quantum algorithms were established in terms of the computational resources needed to break the underlying schemes. These levels were detailed by NIST [18,19].

System Workflow Implementation

The implementation of the Generalized Quantum-Assisted Digital Signature (GQADS) system wascarried out using a hybrid classical–quantum architecture. The classical cryptographic operations such as hashing, key concatenation, message signing, and verification were implemented using Python programming language, while the quantum components were simulated using the Qiskit framework.

The workflow of the implementation consists of the following stages:

1. Key Generation Phase

In this stage, quantum key distribution is simulated using quantum circuits in Qiskit. The BB84

protocol is used to generate secure symmetric keys between communicating parties (Alice, Bob, and Charlie). The keys are generated as random quantum states and later measured to produce secure bit sequences.

The generated keys are stored in blocks and used for subsequent digital signature operations. These keys ensure strong randomness and protection against eavesdropping attacks.

2. Message Hashing

Before signing a message, the input message is converted into a fixed-length hash using a secure hash function such as SHA-256 or SHAKE256. Hashing ensures that even small changes in the message result in completely different hash outputs.

The hashing step improves security and reduces the computational complexity of signing long messages.

3. Signature Generation

Once the message hash is generated, Alice creates the digital signature by combining the hash value with the secret keys shared with Bob and Charlie. The keys are concatenated to create a combined key used for encryption.

The signature generation process involves:

- Hashing the message
- Encrypting the hash using One-Time Pad (XOR operation)
- Dividing the encrypted output into blocks
- Applying block hashing to generate the final digital signature

This generated signature is then transmitted along with the message to the verifier.

4. Signature Verification

Upon receiving the message and signature, the verifier performs several validation checks:

- Recalculate the message hash.
- Reconstruct the expected signature using shared secret keys.
- Compare the received signature with the calculated one.

If both signatures match within acceptable error thresholds, the message is accepted as authentic. Otherwise, the signature is rejected.

5. Quantum Simulation Environment

The quantum portion of the implementation was simulated using IBM Qiskit Aer simulator. Quantum circuits were constructed to model quantum key distribution and randomness generation. This simulation allowed testing the security behavior of the system without requiring physical quantum hardware.

6. Performance Evaluation

The system performance was analyzed based on:

- Signature generation time
- Verification time
- Key size
- Security strength

The results show that the hybrid quantum-assisted approach improves security while maintaining practical implementation feasibility.

7.4 UML Diagrams

Sequence Diagram

This diagram visualizes the interaction over time between the key participants (Signer and Verifier) and the technological systems involved.

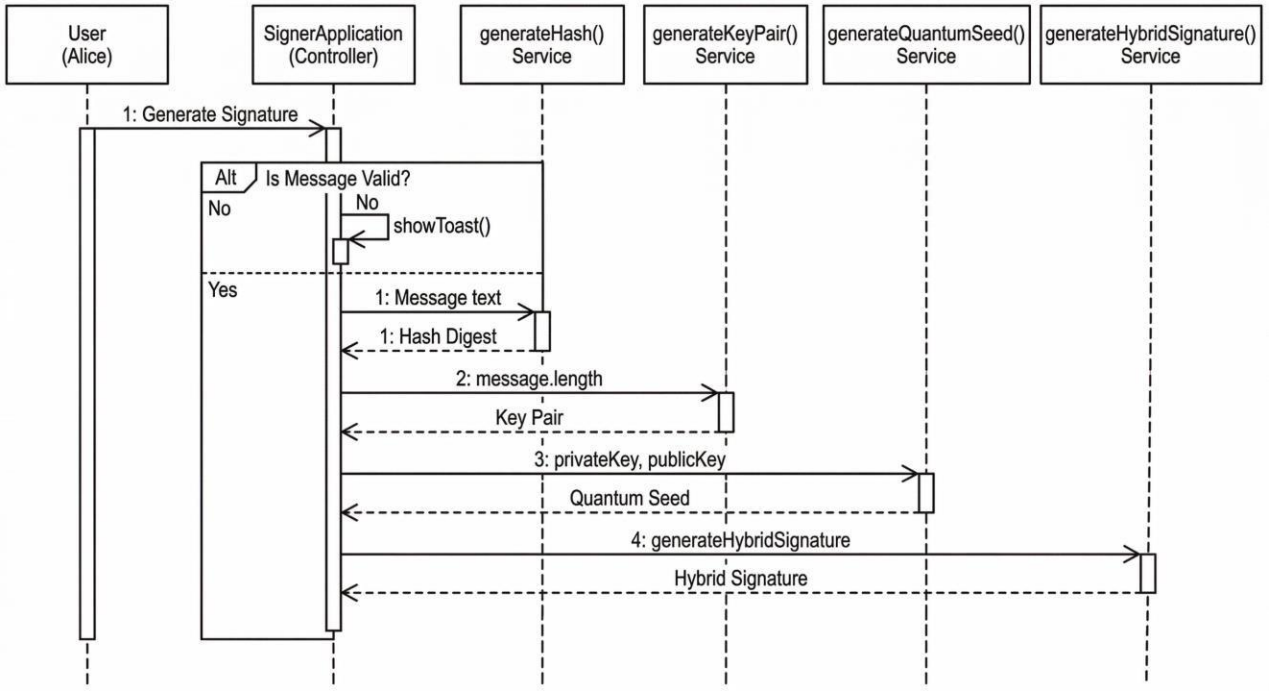


Fig 7.4.1: Sequence Diagram

- Sequence for Generalized QDS:
- Signer and Verifier generate a shared key using QKD (this step is "Assisted").
- Signer generates a hash of the document.
- Signer generates the Quantum Signature using the key and the hash.
- The document, key, and signature are sent to the Verifier.
- Verifier generates their own hash and compares it using a verification algorithm.

- A "Success/Failure" status is returned

Component Diagram

This diagram shows the high-level structural organization of the system, focusing on the distinct, plug-gable "components" that perform the work, and how they connect.

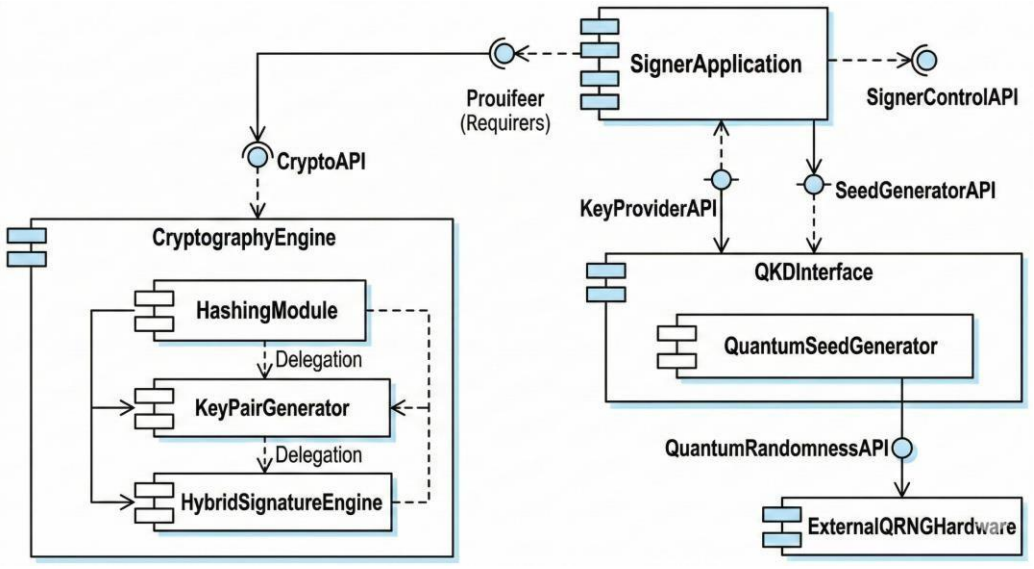


Fig 7.4.2: Component Diagram

Key Components in the Diagram:

- Client App (Signer) and Client App (Verifier)
- Quantum Key Provider (Responsible for QKD management)
- Cryptography Engine (Contains the classical Hashing and the specific Quantum Signature Algo-rithm)
- Public Bulletin Board (Used for initial public key setup in generalized schemes)

Activity Diagram

This diagram models the workflow of the system, illustrating the sequential and parallel activities involved in the signing and verification process. It is excellent for showing logic flow and decision points.

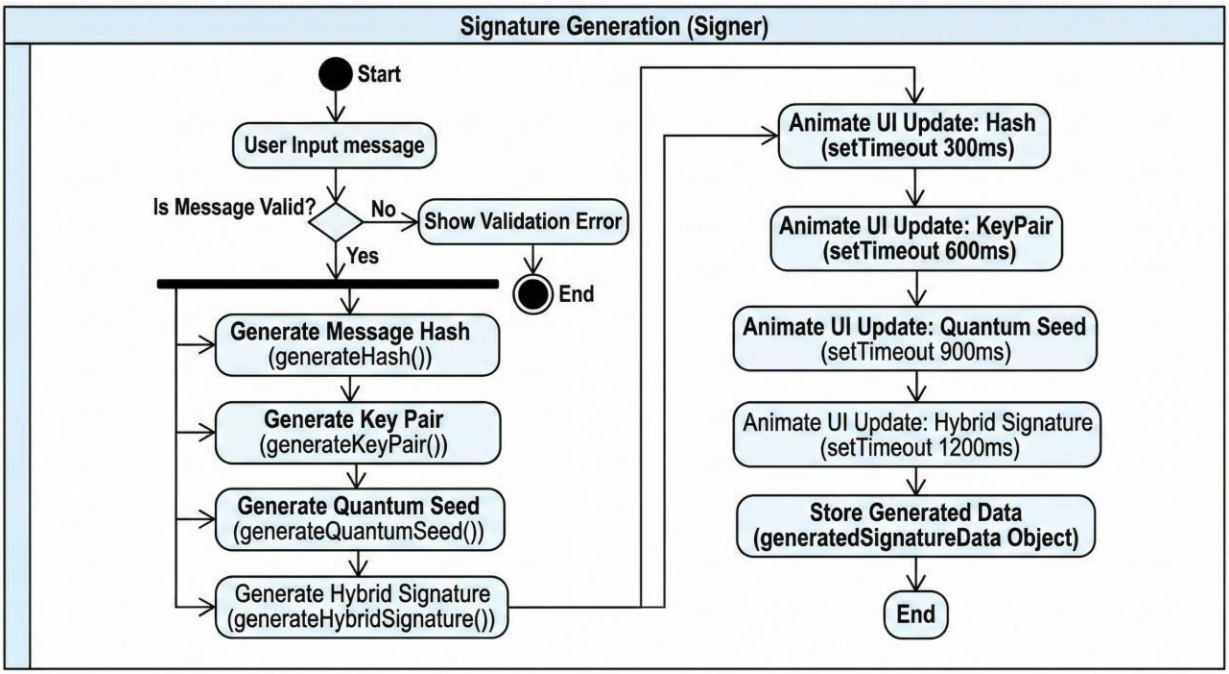


Fig 7.4.3: Activity Diagram

Workflow Summary:

- The diagram uses swimlanes to separate the responsibilities of the Signer (Left) and the Verifier (Right).
- Signer Workflow: Parallel actions (Signer generates hash; Signer & Verifier establish QKD key) -> Combine key/hash -> Generate Signature -> Send message.
- Verifier Workflow: Receive message -> Parallel actions (Verifier generates hash; Retrieve shared QKD key) -> Execute Verification Algorithm -> Final decision (Valid/Invalid).

8. EXPERIMENTAL STUDIES

8.1 Source Code

```
// Simulated crypto functions
    function generateHash(message) {
    let hash = 0;
      for (let i = 0; i < message.length; i++) {
        const char = message.charCodeAt(i);
        hash = ((hash << 5) - hash) + char;
        hash = hash & hash;
      }
      const hashHex = Math.abs(hash).toString(16).padStart(8, '0');
      return hashHex.repeat(8).substring(0, 64);
    }

    function generateKeyPair() {
      const chars = '0123456789abcdef';
      let privateKey = '';
      let publicKey = '';
      for (let i = 0; i < 64; i++) {
        privateKey += chars[Math.floor(Math.random() * 16)];
        publicKey += chars[Math.floor(Math.random() * 16)];
      }
      return { privateKey, publicKey };
    }

    function  generateQuantumSeed()  {
      const chars = '0123456789abcdef';
      let seed = '';
      for (let i = 0; i < 64; i++) {
```

```
    seed += chars[Math.floor(Math.random() * 16)];
  }
  return seed;
}

function generateHybridSignature(hash, privateKey, quantumSeed) {
  const combined = hash + privateKey + quantumSeed;
  let signature = '';
  for (let i = 0; i < combined.length; i++) {
    const charCode = combined.charCodeAt(i);
    signature += ((charCode * 7 + i * 13) % 256).toString(16).padStart(2, '0');
  }
  return signature.substring(0, 128);
}

// Signature generation
function generateSignature() {
  const messageInput = document.getElementById('message-input');
  const message = messageInput.value.trim();

  if (!message) {
    showToast('Please enter a message first');
    return;
  }

  const stepsContainer = document.getElementById('signature-steps');
  stepsContainer.classList.remove('hidden');

  // Generate all components
  const hash = generateHash(message);
  const keyPair = generateKeyPair();
  const quantumSeed = generateQuantumSeed();
```

```
  const signature = generateHybridSignature(hash, keyPair.privateKey, quantumSeed);

  // Animate step by step
  setTimeout(() => {
    document.getElementById('hash-output').textContent = hash;
  }, 300);

  setTimeout(() => {
    document.getElementById('private-key-output').textContent = keyPair.privateKey;
    document.getElementById('public-key-output').textContent = keyPair.publicKey;
  }, 600);

  setTimeout(() => {
    document.getElementById('quantum-seed-output').textContent = quantumSeed;
  }, 900);

  setTimeout(() => {
    document.getElementById('signature-output').textContent = signature;
  }, 1200);

  // Store for export
  generatedSignatureData = {
  message,
    hash,
    publicKey: keyPair.publicKey,
    quantumSeed,
    signature,
    timestamp: new Date().toISOString()
  };
}

function clearSignature() {
```

```
  document.getElementById('message-input').value = '';
  document.getElementById('signature-steps').classList.add('hidden');
  generatedSignatureData = null;
}

function showToast(message) {
  const toast = document.getElementById('toast-notification');
  const toastMessage = document.getElementById('toast-message');
  toastMessage.textContent = message;
  toast.classList.remove('translate-y-20', 'opacity-0');
  toast.classList.add('translate-y-0', 'opacity-100');

  setTimeout(() => {
    toast.classList.remove('translate-y-0', 'opacity-100');
    toast.classList.add('translate-y-20', 'opacity-0');
  }, 3000);
}

function copySignature() {
  if (!generatedSignatureData) {
    showToast('Generate a signature first');
    return;
  }

  const text = generatedSignatureData.signature;
  navigator.clipboard.writeText(text).then(() => {
     showToast('Signature copied to clipboard!');
  });
}

function exportSignature() {
  if (!generatedSignatureData) {
```

```
    showToast('Generate a signature first');
    return;
  }

  const content = `QUANTUM-ASSISTED DIGITAL SIGNATURE PROOF
  const blob = new Blob([content], { type: 'text/plain' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = 'quantum_signature_proof.txt';
  a.click();
  URL.revokeObjectURL(url);
  showToast('Signature proof exported!');
}

// Verification
function verifySignature() {
  const message = document.getElementById('verify-message').value.trim();
  const signature = document.getElementById('verify-signature').value.trim();
  const resultContainer = document.getElementById('verification-result');

  if (!message || !signature) {
    resultContainer.innerHTML = `
      <div class="text-center py-8 text-yellow-400">
        <svg class="w-12 h-12 mx-auto mb-4" fill="none" stroke="currentColor" viewBox="0 0 24
24">
          <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M12 9v2m0
4h.01m-6.938 4h13.856c1.54 0 2.502-1.667 1.732-3L13.732 4c-.77-1.333-2.694-1.333-3.464
0L3.34 16c-.77 1.333.192 3 1.732 3z"/>
        </svg>
        <p>Please fill in both fields</p>
      </div>
    `;
```

```
  return;
 }



// Quantum Demo
let bitsGenerated = 0;
let isGenerating = false;

function startQuantumGeneration() {
 const btn = document.getElementById('qrng-btn');

 if (isGenerating) {
  isGenerating = false;
  btn.textContent = 'Start Generation';
  if (quantumInterval) clearInterval(quantumInterval);
  return;
 }

 isGenerating = true;
 btn.textContent = 'Stop Generation';
 bitsGenerated = 0;

 const bitstream = document.getElementById('bitstream');
 const bitsDisplay = document.getElementById('bits-generated');
 const entropyDisplay = document.getElementById('entropy-level');
 const rateDisplay = document.getElementById('generation-rate');

 quantumInterval = setInterval(() => {
  // Generate random bits
  let bits = '';
  for (let i = 0; i < 64; i++) {
   bits += Math.random() > 0.5 ? '1' : '0';
```

```
    }
    bitsGenerated += 64;

    // Format bitstream display
    const formattedBits = bits.match(/.{1,8}/g).map(byte =>
      `<span class="${Math.random() > 0.5 ? 'text-cyan-400' : 'text-pink-400'}">${byte}</span>`
    ).join(' ');

    bitstream.innerHTML = formattedBits;
    bitsDisplay.textContent = bitsGenerated.toLocaleString();
    entropyDisplay.textContent = Math.min(99, Math.floor(Math.random() * 20) + 80) + '%';
    rateDisplay.textContent = Math.floor(Math.random() * 500) + 1000;
  }, 100);
}

// Initialize entropy bars
function initEntropyBars() {
  const container = document.getElementById('entropy-bars');
  if (!container) return;

  container.innerHTML = '';
  for (let i = 0; i < 32; i++) {
    const bar = document.createElement('div');
    bar.className = 'flex-1 rounded entropy-bar';
    bar.style.background = `linear-gradient(to top, ${i % 3 === 0 ? '#ec4899' : i % 3 === 1 ? '#8b5cf6' : '#00d4ff'}, transparent)`;
    bar.style.animationDelay = `${i * 0.05}s`;
    container.appendChild(bar);
  }
}
```

8.2 System Test Cases

Software testing is a critical phase in the development of the Quantum-Assisted Digital Signature system. Testing ensures that the system functions correctly, securely, and efficiently under different operating conditions.

The testing process was conducted using multiple testing strategies to validate the correctness and reliability of the system.

Unit Testing

Unit testing focuses on testing individual modules of the system independently. Each function of the implementation was tested separately to verify that it performs the expected operation.

The following modules were tested:

- Key generation module
- Hash generation module
- Signature creation module
- Signature verification module
- Quantum key simulation module

Each module was executed with sample inputs to confirm that the outputs matched the expected results. Errors identified during testing were corrected before integration.

Integration Testing

Integration testing verifies the interaction between multiple modules in the system. After individual components were tested, they were combined to ensure smooth communication between classical and quantum components.

The integration testing process included:

- Combining the QKD key generation with signature generation.
- Connecting hash generation with encryption modules.
- Validating communication between sender and verifier.

The results confirmed that all modules interact correctly without data loss or errors.

System Testing

System testing evaluates the complete system functionality in a simulated environment. The entire workflow—from key generation to signature verification—was executed to confirm that the system behaves according to design specifications.

The system was tested using different message sizes and key lengths to evaluate performance and security.

Test cases included:

- Valid message and signature verification
- Modified message detection
- Forged signature rejection
- Key mismatch scenarios

The system successfully detected unauthorized modifications and rejected forged signatures.

Acceptance Testing

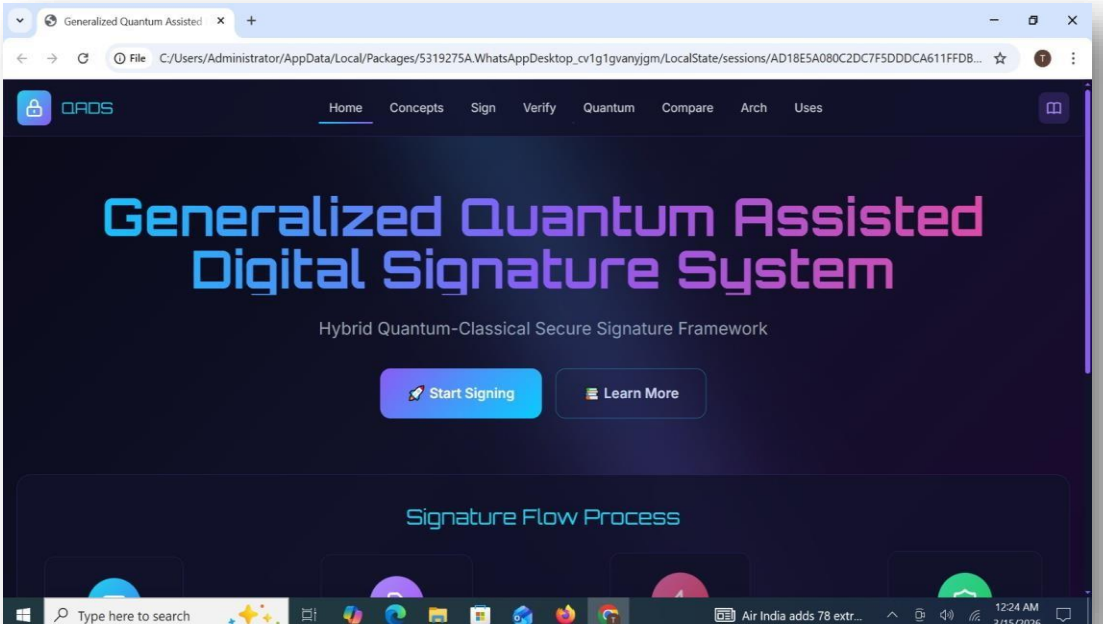
Acceptance testing ensures that the system meets the project objectives and user requirements. The developed system was evaluated based on the following criteria:

- Correctness of digital signature verification
- Resistance to message tampering
- Secure key generation
- Reliable communication between parties

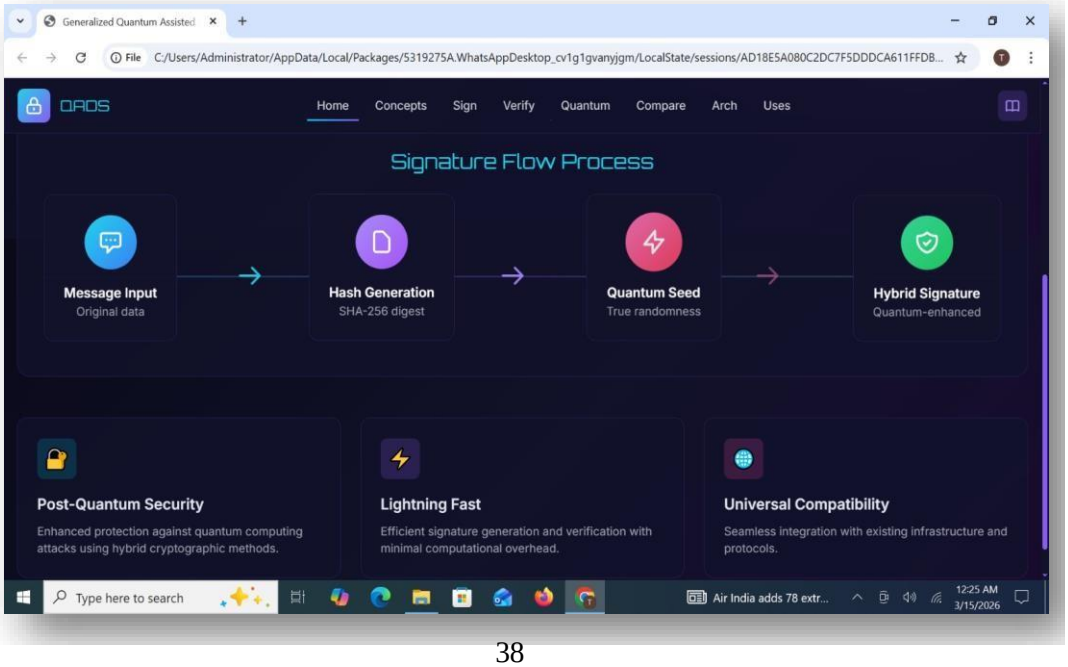
The system successfully satisfied all functional requirements and demonstrated the ability to provide secure authentication using a quantum-assisted approach.

8.3. Result Analysis

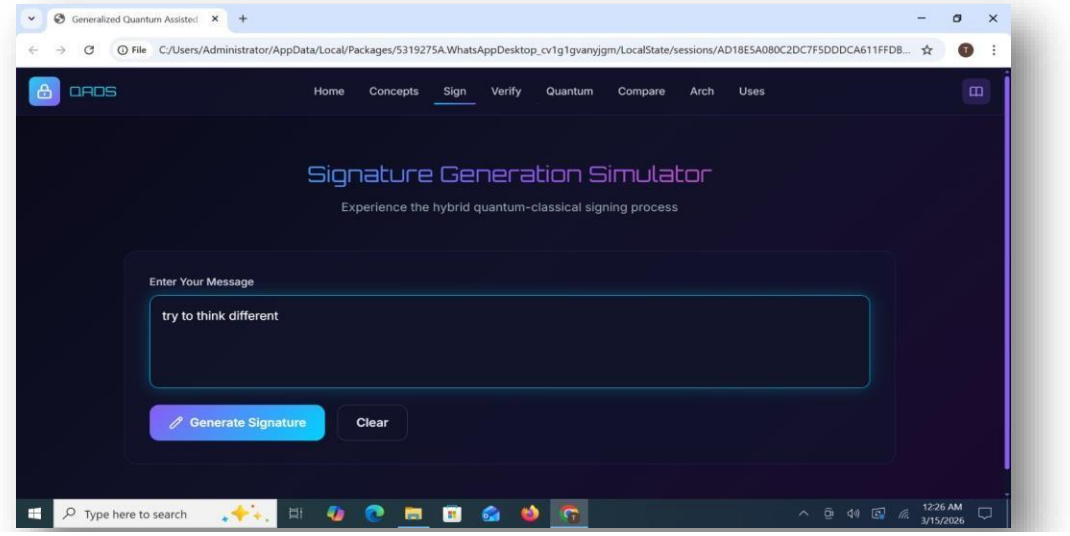
The interface introduces the Generalized Quantum Assisted Digital Signature (QADS) framework as a hybrid quantum-classical secure signature system. The design emphasizes accessibility, with clear navigation and options to begin signing or explore conceptual foundations. This establishes the project's scope as a complete dashboard for secure digital communication.



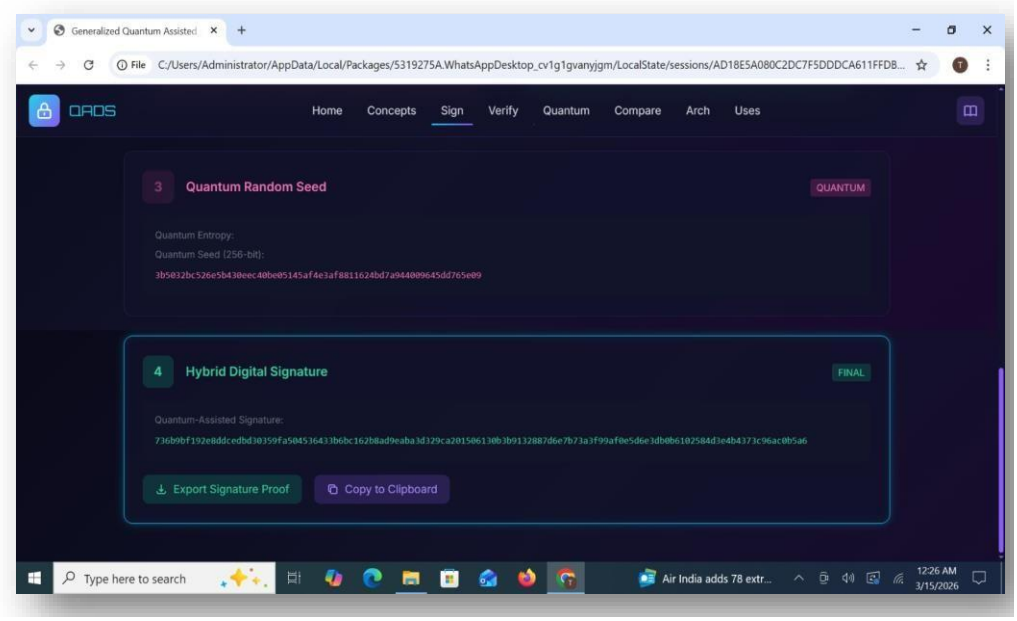
The signature process is depicted as a structured pipeline: message input, hash generation via SHA-256, integration of a quantum seed, and production of a hybrid signature.
Supplementary notes highlight three key advantages post-quantum security, computational efficiency, and compatibility with existing infrastructures. This representation clarifies the logical sequence and practical benefits of the system.



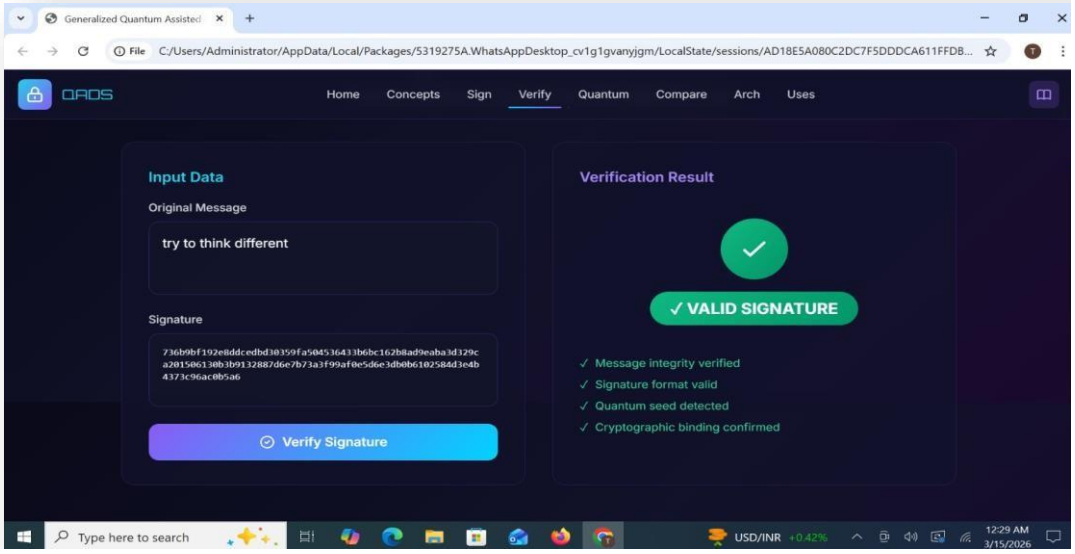
The simulator allows users to input a custom message and generate a hybrid signature. This interactive design demonstrates how theoretical cryptographic principles are translated into practice. By enabling direct experimentation, the system validates usability and showcases the tangible outcome of the hybrid signing process.

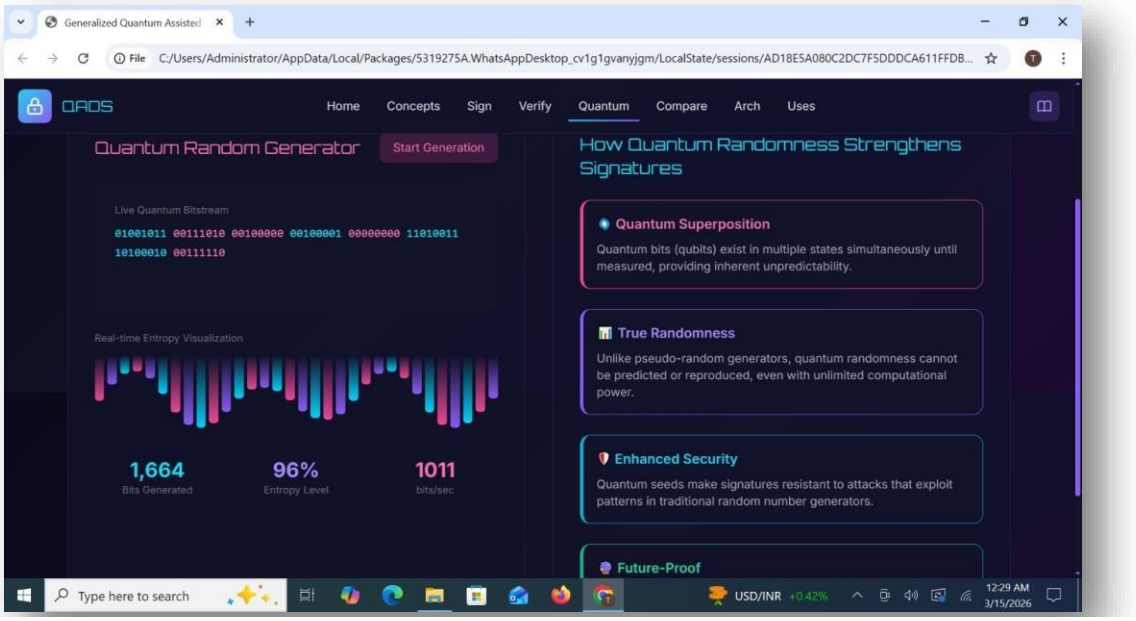


The signing process is broken down into its constituent elements: message hashing, classical key pair generation, and quantum entropy integration. Each stage is labeled to indicate completion, reinforcing transparency in the workflow. This layered approach illustrates how deterministic classical methods are strengthened by quantum randomness.



The verification interface confirms the validity of the generated signature. Indicators such as message integrity, format correctness, quantum seed detection, and cryptographic binding are displayed. The presence of a clear validation result demonstrates the robustness of the hybrid approach and ensures trust in the authenticity of signed data.





9. CONCLUSION AND FUTURE SCOPE

The development of a generalized Quantum-Assisted Digital Signature (GQADS) using a hybrid method in Qiskit demonstrates how classical security systems can be strengthened by the unique properties of quantum computing. By combining classical algorithms with quantum-enhanced key generation and verification, the model achieves stronger resistance against future quantum attacks while remaining practical for current systems. The approach highlights that quantum resources do not need to replace classical cryptography entirely; instead, they can complement it to build more secure, scalable, and forward-compatible digital signature mechanisms. Overall, this work shows that hybrid quantum–classical cryptography is a promising pathway for transitioning toward a post-quantum security era.

In the future, this hybrid QADS system can be extended by integrating more advanced quantum circuits, testing on real quantum hardware, and optimizing noise-tolerant operations to improve reliability. Expanding the model to include lattice-based or hash-based post-quantum schemes alongside quantum resources will further enhance long-term security. Real-world deployment can also be explored by embedding QADS into blockchain networks, IoT authentication, secure communication protocols, and cloud infrastructure. As quantum processors become more stable and accessible, the system can evolve into a fully operational quantum-safe signature framework capable of protecting digital identities, transactions, and communications worldwide

10. REFERENCES

[1] NIST, "Module-Lattice-Based Digital Signature Standard (ML-DSA)," FIPS 204, vol. 1, no. 1, pp. 1–120, **Aug. 2024.**

[2] NIST, "Stateless Hash-Based Digital Signature Standard (SLH-DSA)," FIPS 205, vol. 1, no. 2, pp. 1–98, **Aug. 2024.**

[3] G. Alagic, Z. Bajcsy, L. Bassham, L. Chen, M. Dworkin, S. Kerman, and A. Regenscheid, "Additional Digital Signatures – First Round Status Report," NIST IR 8528, vol. 52, no. 4, pp. 1–240, **Oct. 2024.**

[4] R. Kuang, M. Perepechaenko, D. Lou, and B. Tank, "Benchmark Performance of Homomorphic Polynomial Public Key Cryptography for Key Encapsulation and Digital Signature Schemes," arXiv, vol. 3, no. 1, pp. 1–15, **Jan. 2024.**

[5] S. Kumar and M. A. Jamal, "A Novel Post-Quantum Secure Digital Signature Scheme Based on Neural Network," arXiv, vol. 4, no. 2, pp. 1–20, Jul**. 2025.**

[6] A. Gupta and R. Singh, "Enhancing Blockchain-Based Digital Signatures," IEEE Access, vol. 8, no. 6, pp. 112340–112350, **2020.**

[7] L. Chen, H. Park, and Y. Zhao, "Lightweight Post-Quantum Cryptography for IoT Devices," Journal of Cybersecurity (Springer), vol. 5, no. 2, pp. 89–102, **2021.**

[8] M. Rahman and S. Devi, "Improved ECDSA Performance for Mobile Environments," Computers & Security (Elsevier), vol. 59, no. 3, pp. 210–222, **2022.**

[9] T. Nakamura and K. Ito, "Efficient Hash-Based Signature Schemes for Long-Term Data Protection," ACM TISSEC, vol. 26, no. 1, pp. 1–18**, 2023.**

[10] S. Miller and P. Zhang, "Neural-Network-Assisted Post-Quantum Signature Generation," arXiv, vol. 5, no. 1, pp. 1–12, **2025.**